

Introduction

The StockShark project is a web-based application built to empower users with real-time stock market data and robust portfolio (watchlist) management. The application enables users to register, log in, view interactive charts comparing historical stock data, and manage their personal watchlists by aggregating information from external APIs. Developed using Java Servlets, JSP, MySQL (via XAMPP), and deployed on Apache Tomcat, with real-time charts powered by Chart.js; StockShark was designed to directly address key user stories focused on ease of use, timely access to market data, and streamlined portfolio management.

As the project leader, I was responsible for overseeing the entire project—from initial research and architectural planning to ensuring proper team coordination and documentation. This leadership role was critical in shaping our overall approach and guaranteeing that we met our superior objective: to design a system using professional software architecture and design principles.

Design Process

Our design process was driven by our need to meet detailed user stories while adhering to professional software engineering standards. We prioritized a modular, scalable architecture that allowed for easy extension and maintenance without sacrificing performance.

Clean Architecture Principles:

- **Single Responsibility Principle (SRP):**

  Each class in the StockShark project is designed to have one responsibility. For example, servlets only manage HTTP requests/responses, while separate classes handle business logic and database interactions. This separation ensures that changes in one area (such as the API integration) do not impact others (e.g., user authentication).

- **Open/Closed Principle (OCP):**

  The system is built to be extendable without modifying existing code. For instance, new features can be added by extending base classes or interfaces rather than altering core logic. This allows us to implement future enhancements such as advanced chart interactivity and AI-based predictions without impacting current functionality.
- **Liskov Substitution Principle (LSP):**

  All subtypes and implementations are interchangeable with their base types. This means that classes implementing interfaces (like our Data_Handling_Component) can be substituted without altering the correctness of the program. This facilitates unit testing and future maintenance.
- **Interface Segregation Principle (ISP):**

  Clients are not forced to depend on methods they do not use. Interfaces in the project (for example, those defining user management functions and data handling operations) are focused and minimal. This ensures that each component only implements what is necessary, enhancing readability and maintainability.

- **Dependency Inversion Principle (DIP):**

  High-level modules do not depend on low-level modules; both depend on abstractions. The design uses abstract interfaces for key components (e.g., for database operations and API services), allowing us to change implementations without modifying the core business logic. This increases flexibility and supports unit testing by allowing dependency injection.

Component Specification and Reusable UI Components:

To manage the complexity of StockShark, we began by decomposing the overall system into clearly defined, high-level compound components. Identifying key modules such as the User Management, Stock Data Handling, Portfolio Management, and Chart Visualization components.

This early process was supported by thorough research on industry benchmarks like Yahoo Finance. The result was a comprehensive component specification diagram. This diagram served as an architectural map, outlining the interfaces and interactions between modules

while ensuring that each component fulfilled a specific responsibility. This upfront specification was crucial for maintaining a clean separation of concerns as the project evolved and it provided a solid foundation for scaling and future enhancements.

In parallel with our architectural planning, we focused on the design and implementation of reusable UI components. Recognizing that consistent, intuitive interfaces are key to fulfilling user stories, we developed a library of common UI elements such as login forms, registration screens, interactive charts, and portfolio tables. Each component was designed to be modular and self-contained, facilitating rapid prototyping and ensuring uniformity across the application.

Architectural Concepts:

We adopted a five-tiered architecture: Client-Side Interface (JSP/HTML/CSS/JS), Business Logic (servlets and compound components), Data Access (Database Client using JDBC), External Integration (API calls to Alpha Vantage), and the Database (MySQL via XAMPP). Additionally, we implemented MVC to cleanly separate our presentation layer from controllers and back-end logic.

Project Management Setup:

Our GitHub configuration included using a repository with branch protection and pull request reviews to ensure code quality. Our project board was set up on GitHub Projects to track sprint tasks, bugs, and enhancements. We carried out extensive research on similar projects (e.g., Yahoo Finance and various online stock analysis tools) to help identify key features and define our project scope.

Individual Perspective

For me, the most valuable aspect of the design process was component design. As our codebase grew in complexity, thinking in terms of individual reusable components acted as a map—it helped me navigate intricate system interactions and maintain clarity in my code.

Breaking down the system into loosely coupled modules (such as the Database Client, User Management components, and Data Handling components) allowed me to isolate complex logic and debug more efficiently.

UI Element Decisions: Choosing how our UI elements should look was straightforward, thanks to extensive reference material available from sites like Google Finance. This provided a wealth of inspiration, streamlining my design decisions and enabling rapid prototyping.



Team Perspective

From the team's standpoint, our success stemmed from effective sharing of responsibilities and clear, ongoing communication—principles that were central to our agile methodology. We leveraged GitHub for version control, using pull requests and shared project boards to facilitate comprehensive code reviews and real-time task tracking. Our regular stand-ups ensured that every team member remained aligned with our overall objectives. Although our collaborative efforts were strong, I believe that implementing automated unit testing via GitHub would have further minimized manual review cycles and enhanced our code stability, thereby accelerating our overall delivery process.

Development

The development phase was marked by rigorous technical problem-solving and iterative enhancements based on ongoing feedback. My personal involvement extended across the entire project, ensuring the implementation of our high-level architectural concepts while guiding the team in key areas.

Key Contributions and Insights

- **Leadership and Technical Oversight:**
  I led the project's architectural planning and coordinated the implementation effort, ensuring that every technical decision adhered to Clean Architecture principles. My responsibilities spanned the user-management, portfolio, and watch-list modules, as well as external-API integrations and all database interactions.

  To keep each sprint on schedule, I established a reliable development environment: after analysing candidate technologies, I integrated Jakarta EE, Apache Tomcat, and Chart.js with IntelliJ IDEA Ultimate and a MySQL instance provisioned through XAMPP. With each new technology I sourced relevant training material and documentation for the team to reduce individual learning curves.



- **Coding and Debugging:**
  I personally implemented the critical modules specified in the Compound component diagram, meticulously migrating the team's existing code to conform to the new architectural specification.

  Although each member was assigned distinct wireframes, my deeper knowledge of the codebase broadened my remit: I directed the implementation of both frontend elements and middleware, including the Compare Stocks integration, the Portfolio and Watchlist servlets, and the Database Client methods that handle external-API calls and stock-data parsing.

  Throughout, I established robust error-handling and logging practices, using targeted debug statements to trace the complete data flow from the API layer to the Chart.js visualisations.

- **GitHub and Project Board Configuration:**
  I configured our GitHub infrastructure end-to-end: created the central repository as the single source of truth for version control, established branch-protection rules and a disciplined branching strategy, and set up the Projects view with a Kanban project board for agile task management. Together, these measures give the team transparent, real-time insight into task status and code changes, ensuring traceable, dependable progress throughout the development lifecycle.

- **Research and Feature Identification:**
  I directed extensive research on industry-standard solutions and similar projects. This informed our scope, feature set, and architectural decisions; ensuring that our designs were both user-centric and technically robust.

What Went Well:

I was pleased with the level of collaboration and the adoption of agile practices that made the coding process structured and iterative. Our code reviews ensured high-quality code and fostered team learning.



Challenges:

Integrating external API data with our internal database schema proved challenging, particularly due to the API's strict limit of 25 requests per day. Debugging integration issues and ensuring data consistency required numerous iterations of testing and careful refinement of our error-handling strategies. This constraint forced us to be deliberate in our approach to managing exceptions and logging each step of the process with catch err and console prints. This generally slowed the development process particularly in the plans for AI analysis which requires frequent API data to train the model.

Areas for Improvement:

In hindsight, while I was deeply involved in every aspect of the project, I would have liked to automate more responsibilities particularly around writing unit tests so that I could focus more on strategic architectural decisions.

Lessons Learned:

This project enhanced my skills in full-stack development, API integration, and agile project management. I learned that clear architectural design and thorough research are critical to managing complex development tasks and aligning them with user stories.

Teamwork

How well do you think you worked as a team?

We worked well as a team by maintaining continuous and open communication. Regular stand-ups, agile planning sessions, and an organized GitHub project board enabled everyone to stay aligned on goals and quickly address issues. By leveraging pull requests for thorough code reviews, we ensured that every contribution was scrutinized, promoting a high standard of quality while reducing integration issues.

What did you learn about teamwork from this project?

This project reinforced the importance of clear communication, structured task allocation, and regular feedback. I learned that maintaining a balance between individual ownership and collaborative problem solving is key to a successful project outcome. Our agile approach



allowed us to adjust strategies on the fly, while our disciplined use of version control and project management tools ensured transparency and accountability. These practices not only improved our technical output but also fostered a supportive environment where team member's strengths were fully utilized.

What percentage of the overall work for the project would you estimate that you personally did?

As project leader, I directed the overall system architecture and coordinated design decisions. Each team member contributed based on their strengths: from user management and portfolio implementation to UI design and test documentation.

Reflection on Overall Learning and Skill Growth

The project has significantly enhanced both my technical abilities and leadership skills. It confirmed the critical role of professional software architecture in managing complex systems.

Technical Growth:

I have improved my expertise in full-stack Java development, particularly in integrating external APIs and managing database operations through a robust Database Client. My understanding of architectural principles such as SRP, OCP, and DIP has deepened, making my code more modular and maintainable.

Process and Collaboration:

The use of agile methodologies, combined with GitHub project management and pull requests, has taught me effective collaboration strategies and enhanced my appreciation for individual contributions and collective decision-making.

Challenges and Future Focus:

I would have liked to complete the non-functional requirement regarding the AI analyzer: an auxiliary feature that would have provided dynamic market predictions. In future projects, I plan to implement automated unit testing to reduce manual code reviews, enabling more focus on feature enhancements like dynamic API provider handling. This would allow the system to autonomously switch between providers when API request limits are reached, further enhancing the user experience and propel the development of our AI model.